

Reviewer Pack — Minimal Rank of K-Theory over Tensor Network Valuations

Entry 9f7ef09f92b6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:13:28 UTC

Minimal Rank of K-Theory over Tensor Network Valuations

Entry ID: 9f7ef09f92b6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 13:13:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (Groups) **Field B** (complexity object): Communication Complexity (Tensor Network)

Statement:

[‘For a given Boolean function f , the minimal rank of its associated tensor network valuation in the category of modules over the algebra generated by f is polynomially related to the communication complexity of f .’, ‘Specifically, there exists a constant c such that for all boolean functions f , the minimal rank $R(f)$ of its tensor network valuation satisfies $R(f) \leq c * CC(f)$, where $CC(f)$ is the communication complexity of f .’, ‘This relationship holds with equality if and only if f can be computed by a constant-depth unbounded fan-in circuit.’]

Rationale (proposer’s reasoning):

[‘Algebraic K-Theory provides a framework for studying the classification of modules over an algebra, which may provide a new perspective on understanding the complexity of boolean functions through their tensor network representations.’, ‘Communication complexity is a measure of how much information needs to be communicated between two parties in order to compute a given function. If the minimal rank of a tensor network valuation is polynomially related to communication complexity, it suggests that these two concepts are deeply connected.’]

Taxonomy category: Minimal Rank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4e6539f695f55e55

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all boolean functions f , the ratio of minimal rank $R(f)$ to communication complexity $CC(f)$ is less than or equal to a constant c ($R(f) \leq c * CC(f)$) AND this relationship holds with equality for at least 25% of the seeds. The conjecture is

falsified if any seed produces a ratio greater than c , or if the relationship does not hold with equality for more than 75% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic K-theory AND tensor network valuations AND communication complexity-minimal rank K-theory AND tensor networks AND valuation communication complexity- tensor network valuation polynomial relationship minimal rank K-theory communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return ''.join(random.choice('01') for _ in range(2**n))

    def communication_complexity(f):
        n = len(f)
        # Simplified protocol for demonstration
        return n

    def tensor_network_rank(f):
        # Placeholder function to simulate a rank calculation
        return len(f) * 2

    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    cc_f = communication_complexity(f)
    R_f = tensor_network_rank(f)

    ratio = R_f / cc_f if cc_f != 0 else float('inf')
    conjecture_holds = ratio <= 1.0
```

```

counterexample = "" if conjecture_holds else f"R(f)={R_f}, CC(f)={cc_f}"

return {
    "metric_name": "ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 6)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.75:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"failing_ratio\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, making it impossible to evaluate the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12496
2	propose	ollama_remote	glm4:latest	0	0	10559
3	preregistration	ollama_remote	glm4:latest	0	0	6532
4	novelty	ollama_remote	glm4:latest	0	0	4602
5	novelty	ollama_remote	glm4:latest	0	0	5345
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42161
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6807
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7005
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6600
10	judge	ollama_remote	glm4:latest	0	0	25705

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127813 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9f7ef09f92b6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9f7ef09f92b6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9f7ef09f92b6.tar.gz` (if generated)